
From Primitive to Exploit Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *November 21st, 2019*

Introduction

1. The source auditing lecture's mitigation map names the bypass column (ROP, infoleak, grooming) without building them. This module builds each: what the primitive is, what technique pairs with it, and how the pairing is priced.
2. Scope statement: this is a *recognition-level* treatment — how the technique families work, what they need, why each stops working. Building a real chain end-to-end is an exploitation course in itself.

Control-Flow Reuse: ROP and JOP

1. The NX/DEP row's bypass family. Premise: with the stack non-executable, reuse existing executable bytes instead of injecting code.
2. **Ret2libc**: the trivial chain — overwrite the return address to jump at `system("sh")` (or the platform equivalent) with argument registers/stack pre-filled per the calling convention. One function, no gadget search: the first technique to try against any no-canary stack overflow.
3. **ROP** (Return-Oriented Programming): chain short "gadget" snippets ending in `ret`; each gadget performs a compute step (register move, add, dereference-load). The chain lives on the stack; each return consumes the next gadget address. Bootstrap requirement: a controlled return address and controlled stack contents behind it.
 - (a) Gadget discovery: scan the binary (and loaded libraries) for `ret`-terminated fragments — ROPgadget/ropper class tools; after PIE defeat you can mix libraries.
 - (b) Calling a syscall on x86_64 needs `rax=NR`, `rdi/rsi/rdx` args: typical chains are `load gadget` → `pop gadgets` → `syscall`; the gadget census of the target decides feasibility.
 - (c) Chain fragility: stack alignment (x86 `movaps` traps on unaligned), NULL bytes limiting address choice, and the count of useful gadgets — small static targets may have too few, requiring `ret2libc` or more bugs to get there.
4. **JOP** (Jump-Oriented): same idea with `jmp` dispatch through a register with a dispatcher gadget; survives CFI-in-a-can (`ret`-only policies) and is the standard answer where the toolchain removed `ret`-gadgets specifically.

5. **Stack pivot:** when the initial control is a single indirect call/branch (function-pointer overwrite) rather than a return, first redirect `rsp` into attacker data (`xchg esp, eax` class gadgets), *then* run a ROP chain. Prerequisite chain: controlled call → pivot → chain — why function-pointer hijacks pair with pivots in every writeup.
6. The variants that recur, and the condition each needs:
 - (a) **ret2libc** with argument control: the same as the trivial chain, but the argument registers (`rdi` on x86-64, `x0` on ARM64) must be populated first — so it needs a `pop rdi; ret` gadget and a pointer to `"/bin/sh"`. Without argument control it is `system` called with whatever happens to be in the register.
 - (b) **ret2plt/ret2dlresolve**: call an existing PLT stub directly (no leak needed for the call target), or make the dynamic resolver resolve an attacker-named symbol. `ret2dlresolve` needs control of the relocation structures on the stack and is the no-libc-leak answer on a PIE binary.
 - (c) **ret2csu**: the `__libc_csu_init` gadget populates several argument registers at once from the stack, which is why it is the standard first gadget when no pop gadgets exist.
 - (d) **SROP** (sigreturn-oriented programming): the attacker forges a `sigcontext` on the stack and triggers `rt_sigreturn`, which loads *all* registers at once. It needs only a `syscall` gadget and a controlled stack, which is why it is the standard answer when the gadget census is too small for a normal chain.
 - (e) **BROP/JIT-ROP**: when the binary is not available (remote, or a browser target), leak the code at runtime and build the chain on the fly. This is the reversing module's dynamic tooling applied to exploit construction.
7. The modern blocks and their bypasses: **CET** (shadow stack, `endbr64`) blocks a plain `ret`-redirect unless a legitimate `endbr64` landing pad exists, which pushes toward data-only attacks or `call`-oriented chains; **ARM64 PAC** (`paciasp/autiasp`) blocks overwriting a saved return address unless the PAC value can be forged or the signed pointer replayed; **MTE** blocks tag-mismatched accesses. Read the target's `/proc/cpuinfo` and compiler flags before choosing a technique.
8. Tooling: `ROPgadget/ropper/rp++` for the gadget census, `pwntools ROP` for chain assembly and `one_gadget` for the system-and-nothing-else case. The chain is assembled, then tested, then re-assembled; the debugger (the debugging module) is the loop.

Infoleak: Defeating ASLR Before Any Chain

1. Every ROP-family chain needs absolute addresses. ASLR randomizes them per-boot; the chain therefore needs a leak first — this is why “bug 1 = leak, bug 2 = control” is the standard exploitation shape.
2. Leak primitive families:
 - (a) Reflected OOB reads whose output length/reflection is attacker-influenced (format string reads, HeartBleed-class over-reads, unbounded `snprintf` echoes).

- (b) Uninitialized-memory reads returning stale heap/stack content (the uninitialized-use section's flip side: attacker sprays pointers, then triggers a use that echoes them).
 - (c) Side channels: timing (cache sets, branch predictors), error messages, crash-adjacent dumps, and pointer-value echo through logs.
3. What to leak: one pointer per randomized region (libc for gadgets, stack for pivots, heap for groom-relative addresses). Then the mitigation map's ASLR row collapses for that session.
 4. Pricing a leak: bytes/attempt, retry budget (does the connection survive the read?), and whether the leak's target address survives between sessions (per-boot vs per-process randomization).
 5. The leak sources that recur, by primitive:
 - (a) A reflected over-read: `printf` with an attacker-controlled `%p/%s` argument, an unbounded copy-out, a heartbeat-style over-read (CVE-2014-0160). The bug is a read whose length or target the attacker controls.
 - (b) Uninitialized memory returned to the attacker: a partially filled struct copied out, a freed-then-reused buffer echoed back (the source auditing lecture's uninitialized-use section).
 - (c) Allocator metadata: a `main_arena` pointer or an unsorted `bin fd/bk` read from a corrupted chunk, or a `tcache` entry pointing into `libc`. This is the standard glibc base leak and needs only a relative read.
 - (d) A stack or `environ` pointer: gives the stack base, which is what a SR0P or a stack pivot chain needs.
 - (e) The binary's own base: a partial pointer or a mapping address (`/proc/self/maps` if readable) defeats PIE; on a fully PIE target the leaked `libc` address plus the known link-time offset is often enough.
 6. The leak consumers, one per randomized region: a `libc` leak for the gadgets and system, a stack leak for a pivot or SR0P, a heap leak for the groom's relative addresses, and a binary leak for PIE. One leak usually yields the rest, because the libraries' relative offsets are fixed at link time.
 7. Tooling: `pwntools LibcSearcher` and the `libc-database` identify a `libc` from one leaked symbol's offset; `one_gadget` maps a `libc` build to a single-call chain; ASLR on 32-bit is brute-forceable in a loop, which is why the retry budget above matters.

Allocator Grooming: Making the Primitive Land

1. Heap primitives (sizes/lengths/double-free/UAF) convert into control only if the corrupted allocator state serves an attacker-shaped allocation next. Grooming is the heap-layout side of every UAF/overflow exploit and the direct answer to the source auditing lab's Exercise C open question.
2. glibc's allocator internals (the small program walker's model; versioned — verify against the target's glibc):

- (a) Small allocations go to **tcache** (per-thread, size-binned singly-linked lists; first-fit; a double-free of one tcache entry yields two allocations of the same address — 2.29+ prints “detected double free” on an exact same-size repeat, so shapes must oblige).
 - (b) **fastbins, unsorted bin, small/large bins**: the free-list machinery behind bigger chunks; metadata (fd/bk pointers, size fields) is what a controlled overflow re-writes to force “next allocation serves my target”.
 - (c) The classic shapes: overwrite the size field to overlap chunks (House of Elephant/Force family), corrupt tcache fd to allocate over a function pointer, and unsorted-bin attacks to leak a libc address directly.
 - (d) Conservatism rules: every allocator hardening update (safe-linking in 2.32, tcache key double-free guard in 2.29/2.34, fd-pointer mangling) changed the toolbox — price the *target’s* libc version first.
3. Kernel slab (SLUB) parallels: per-size freelist in page-sized slabs; same grooming logic with different primitives (the kernel lecture’s spray list: msg_msg, pipe buffers, add_key); the cross-cache variant reclaims a freed slab page for a different cache — the kernel form of “groom until the type confusion is served”.
 4. Practical grooming loop (mirrors the debugging lab): break on malloc/free, record a transcript of sizes/order under the target’s own command sequence, replay onto your exploit’s predicted sequence, confirm the landing allocation receives your sprayed data (removing the guesswork one step at a time).
 5. The glibc facts that decide every groom, in the order an exploit needs them:
 - (a) **tcache** (2.26+): per-thread, one bin per size, last-in-first-out, seven entries per bin by default. A free goes to tcache before any other bin, which is why a tcache-sized primitive is the most reliable one. The tunables (glibc.malloc.tcache_count, glibc.malloc.tcache_max) change the arithmetic, so read them.
 - (b) **fastbins**: small chunks with a LIFO singly-linked list; a double free is caught only for the exact same size (fasttop check), so a double free of two sizes is the classic bypass.
 - (c) **unsorted/small/large bins**: freed chunks that are coalesced and sorted; their fd/bk pointers are the leak source and the unlink target.
 - (d) **safe-linking** (2.32+): the freelist fd is stored as (ptr >> 12) ^ pos, so an attacker who overwrites it must know the chunk’s own address — a heap leak is now a precondition for the classic tcache-poisoning groom.
 - (e) **the tcache key** (2.29+): a freed tcache chunk stores a per-thread key, and free refuses a second free of the same chunk (“free(): double free detected”). The workaround is to overwrite the key, which itself needs a write primitive.
 - (f) **the mmap threshold**: allocations above it are served by mmap, not the main arena, which changes which groom applies (M_MMAP_THRESHOLD, glibc.malloc.mmap_threshold).
 6. The named techniques, and the primitive each needs: house of force (overwrite the top chunk size), house of spirit (free a forged chunk pointer), house of einherjar (forge a chunk’s prev_size), unsafe unlink (a chunk whose fd/bk pass the check),

tcache poisoning (overwrite a tcache fd to return an arbitrary address), and the `_IO_FILE` family (FSOP, house of apple) which overwrites a file structure's vtable. Each is the source auditing lecture's allocator-metadata pattern, priced against the target's glibc version.

7. The kernel parallel is the same logic in the SLUB allocator: a per-cache freelist in page-sized slabs, with `CONFIG_SLAB_FREELIST_HARDENED` and `_RANDOM` as the safe-linking analogue, and cross-cache reclaim as the "reuse a freed page for a different object type" technique.

Bypass Puzzle: Pricing a Real Chain

1. Worked pricing of the course's own `heap.c` (its stack layout and allocator are known from the labs): with ASan the double-DELETE is caught immediately (quarantine); with plain glibc the UAF gives a freed-but-pointer-live allocation — groom with same-size CREATES so the re-CREATE lands on the stale slot, then FIND-after-free yields a type-confused read primitive. Each mitigation row changes what the chain needs; walk the map's rows against this one bug.
2. For each finding, write the pricing table: bug class $\times$ active mitigation $\times$ primitive (= chain steps $\times$ required preconditions). The report's "exploitability statement" is exactly this table read aloud.
3. The same table, filled in for the course's own `heap.c` double-DELETE (the source auditing lab's UAF), which is the worked example every report should imitate:

Mitigation	Effect on this bug	What the chain needs
ASan build	double-DELETE caught at once (quarantine)	nothing: the bug is not exploitable under ASan
glibc tcache (2.29+)	double free detected by the key check	overwrite the tcache key first, or free two different sizes
glibc safe-linking (2.32+)	tcache fd is mangled by the chunk address	a heap leak before the poisoning groom
NX / DEP	no shellcode on the stack	a ROP or <code>ret2libc</code> chain after the leak
ASLR / PIE	no fixed addresses	a libc leak (unsorted-bin or <code>_IO_FILE</code>)
FORTIFY_SOURCE	checked <code>strcmp</code> / <code>memcpy</code> paths	use an unchecked path, or a controlled-length read

1. Reading the table: each row is a step in the chain, not a verdict. The exploit is the composition of the rows; the report is the table plus the one-sentence root cause.
2. The general pricing rule, which is the whole module: a primitive is worth what the *weakest* active mitigation lets it do. An arbitrary write on a fully-mitigated target is worth a leak plus a groom plus a chain; the same write on an old target is worth a single `ret2libc`.

Conclusion

The mitigation map's bypass column is not magic: it is four techniques (code reuse, leaks, grooming, pivots) paired to bug classes by need. Auditors who cannot build the pairing systematically underprice their own findings; exploit developers who skip the audit discipline overpay for the same primitives. This module is the handshake between the two — the course's standing bridge from the source auditing lecture's map to the kernel module's elevation targets.

The discipline to take away: for every finding, name the primitive, name the active mitigation, and name the next step. "I have a UAF" is not a finding; "I have a same-size UAF on glibc 2.31, the tcache double-free check is the only obstacle, and I need a heap leak for safe-linking" is. The three steps of every chain are the three sections of this module: get the addresses (infoleak), make the corruption land (grooming), then turn the landing into control (code reuse or a data-only write).

References

1. Return-oriented programming: original papers by Shacham (2007) and Checkoway et al (2010, JOP)
2. pwntools: docs.pwntools.com (gadget search and chain assembly support)
3. ROPgadget / ropper / rp++ (gadget census tools)
4. glibc malloc internals, Marshall (Phrack "Malloc Des Maleficarum") and the glibc malloc wiki writeups
5. how2heap: github.com/shellphish/how2heap (allocator technique drills)
6. The Shellcoder's Handbook (2nd ed) — mitigation-era exploitation chapters
7. SROP: Bosman & Bos, "Framing Signals — a Return to Portable Shellcode" (2014)
8. Data-only and counter-CFI: Hu et al, "Data-Oriented Programming" (2022) and Schuster et al, "Counterfeit Object-Oriented Programming" (2015)
9. Blind and JIT-ROP: Bittau et al (2014) and Snow et al (2013)
10. libc-database and one_gadget (libc.rip, one-gadget)